Académie de Toulouse
Lycée Antoine Bourdelle ,Montauban

Projet de Fin d'Études:

PICO-CENTRALE HYDROÉLECTRIQUE

en vue de l'obtention du

Brevet de Technicien Supérieur de

Cybersécurité ,Informatique et Réseaux ,Électroniques

Spécialité : Électroniques et Réseau

PARTIE 3 : Communication LoRa

Etudiant 3 : Matheis MICHALAK

SOMMAIRE

Contexte : **3**
Figure 1: Synoptique du système 3
Technologies Utilisé : **4**
1. Module E32-900T30D : 4
a. Caractéristiques Principales : 4
b. Modes de Fonctionnement : 4
c. Types de Transmission 5
d. Indicateur AUX : 5
e. Sécurité et Fiabilité 5
2. Schéma de Cablage : 6
a. Configuration des Pins : Phase de Developpement 6
b. Schéma Proteus : Phase de Developpement 6
3. Empreinte Ares : 7
a. Côte de la datasheet : 7
b. Empreinte de Routage : 7
1. Gestion de l'AUX : **8**
1.1. Fonctionnement et Utilité : 8
1.2. Code : 8
2. Gestion des Modes : **9**
2.1. Fonctionnement des Modes : 9
2.2. Gestion de M0 et M1 : 9
2.3. Code : 9
3. Configuration de la Communication : **10**
3.1. Adressage du Module : 10
3.1.1. Principe : 10
3.1.2. Code : 10
3.2. Communication Module a Module : 10
3.2.1. Principe : 10
3.2.2. Code : 11
3.3. Communication par Canal / Broadcast : 11
3.3.1. Principe : 11
3.3.2. Code : 11

Contexte :

Les tâches qui me sont données en tant qu'Étudiant n°3 sont de mettre en place un protocole de communication entre le controleur local et le controleur principal a l'aide d'un module lora (pour echanger sur l'etat du systeme, le niveau de l'eau et les alertes de crue dans la regions et ses alentours.

Bien evidemment ma partie comprend la creation du composant pour le schéma éléctrique ainsi que son empreinte Ares ,du code pour l'etablissement du protocole de communication et les tests associes

L'exigences principal de ma partie est "Communication entre le contrôleur local et le contrôleur principal"

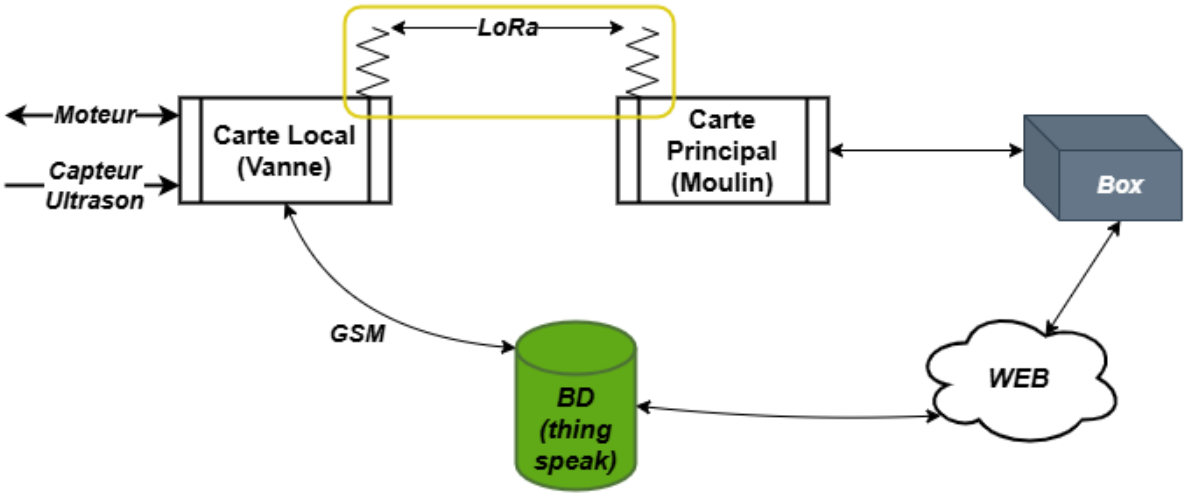


Figure 1: Synoptique du système

Technologies Utilisé :

1. Module E32-900T30D :

Le module **E32-900T30D** est un émetteur-récepteur sans fil à port série (UART). Il utilise la technologie de modulation **LoRa** à spectre étalé, ce qui lui permet d'atteindre des distances de communication relativement élevé tout en offrant une forte immunité aux interférences.

a. Caractéristiques Principales :

- **Distance de transmission :** Peut atteindre jusqu'à 8 km dans des conditions idéales (air libre, antenne gain 5dBi, hauteur 2,5m).
- **Fréquence :** Fonctionne sur la bande 862MHz à 931MHz, supportant les bandes européennes standards 868/915MHz.
- **Puissance d'émission :** Jusqu'à 30 dBm (1W), ajustable par logiciel.
- **Alimentation :** Entre 3,3V et 5,5V DC. Pour une performance optimale (puissance maximale), une tension de 5V ou plus est recommandée.

b. Modes de Fonctionnement :

Le module dispose de quatre modes de fonctionnement déterminés par l'état des broches **M0** et **M1**:

Mode	M1	M0	Description
0 - Normal	0	0	Transmission transparente. Le port série et le canal sans fil sont ouverts.
1 - Wake-up	0	1	Identique au mode normal, mais ajoute un préambule pour réveiller les récepteurs en mode 2.

2 - Power-saving	1	0	Port série fermé. Le module écoute périodiquement le signal de réveil (polling) pour économiser la batterie.
3 - Sleep	1	1	Mode veille pour la configuration. Utilisé pour lire ou modifier les paramètres du module via des commandes HEX.

c. Types de Transmission

Le module supporte deux principaux types d'envoi de données :

- **Transmission transparente (Broadcasting)** : Le module envoie les données à tous les autres modules configurés sur le même canal et avec la même adresse (ou adresse universelle 0xFFFF).
- **Transmission fixe (Fixed-point)** : Permet d'envoyer des données à un module spécifique en spécifiant son adresse et son canal au début du paquet de données.

d. Indicateur AUX :

La broche AUX est utilisée pour indiquer l'état du module:

- Elle passe au niveau **bas (0)** lorsque le module est occupé (auto-test à l'allumage, transmission sans fil en cours ou tampon de données plein).
- Elle passe au niveau **haut (1)** lorsque le module est prêt à recevoir de nouvelles données sur son port série ou que la tâche en cours est terminée.

e. Sécurité et Fiabilité

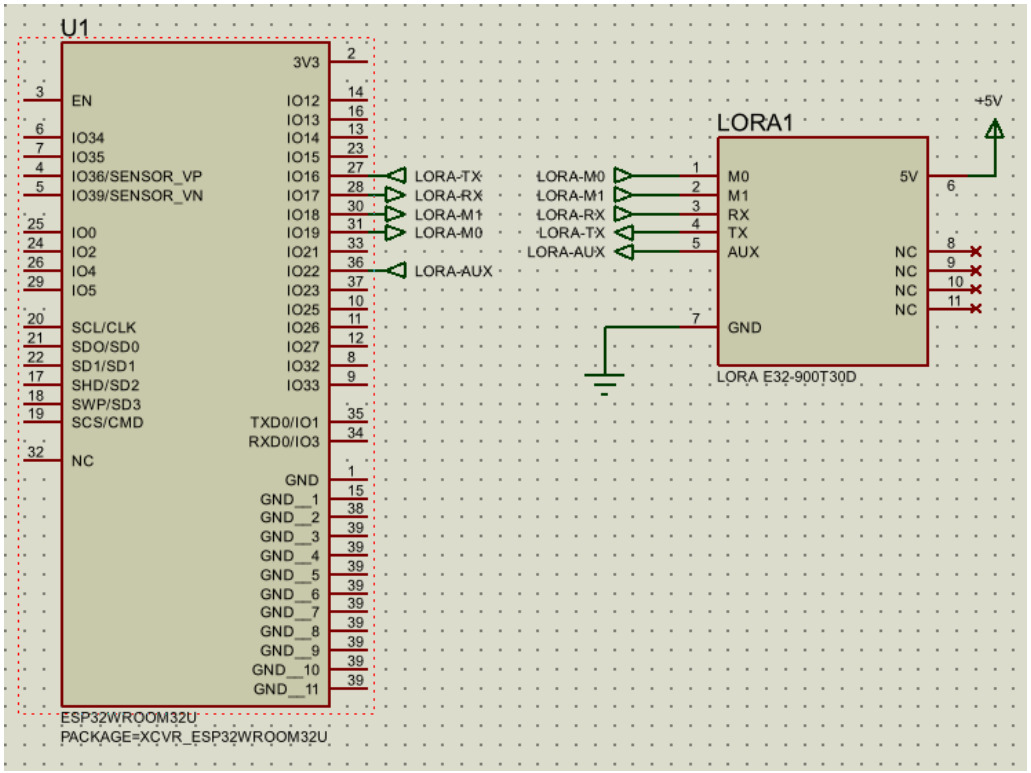
- **FEC (Forward Error Correction)** : Le module intègre une correction d'erreur directe qui améliore la stabilité de la communication en cas d'interférences.
- **Chiffrement et compression** : Les données sont compressées et cryptées avant l'envoi pour assurer la confidentialité et l'efficacité de la transmission.

2. Schéma de Cablage :

a. Configuration des Pins : Phase de Developpement

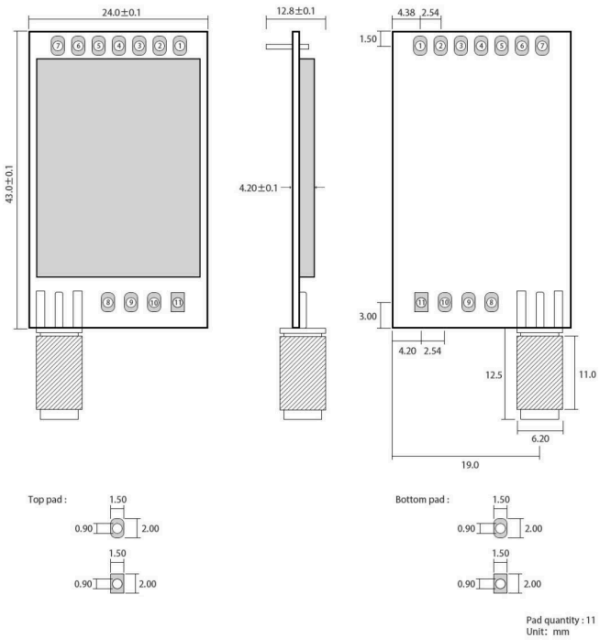
No.	Description	Direction	No.	Description	Direction
1	M0	Entrée	7	GND	Entrée
2	M1	Entrée	8	Fix Hole	NC
3	RX	Entrée	9	Fix Hole	NC
4	TX	Sortie	10	Fix Hole	NC
5	AUX	Sortie	11	Fix Hole	NC
6	VCC	Entrée			

b. Schéma Proteus : Phase de Developpement

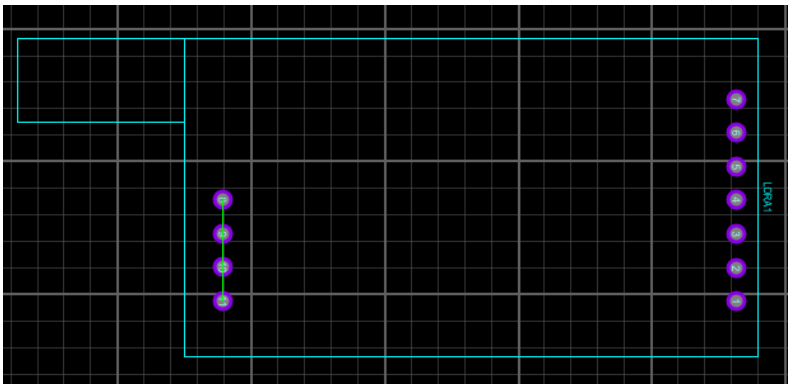


3. Empreinte Ares :

a. Côte de la datasheet :



b. Empreinte de Routage :



1.Gestion de l'AUX :

1.1. Fonctionnement et Utilité :

L'AUX est une information emise par le module et reçu par l'ESP32

Quand l'AUX est a 1 ,le module peut effectuer une action car il ne fait rien pour l'instant

Quand il est a 0 par contre ,le module ne peut pas traiter immediatement l'information envoyé par l'ESP32 car il est encore occupé avec une précedente commande

Sans la gestion de cette information ,le module se trouve surbooké et ne comprend pas se qu'on lui demande ce qui finit par rendre le module inutilisable

1.2. Code :

```
def wait_aux(self):
    while self.aux.value() == 0:
        time.sleep_ms(1)
    time.sleep_ms(2)
```

2. Gestion des Modes :

2.1. Fonctionnement des Modes :

2.2. Gestion de M0 et M1 :

2.3. Code :

```python
def set_mode(self, mode):
    self.wait_aux()
    if mode == 0:     # NORMAL
        self.m1.value(0); self.m0.value(0)
    elif mode == 1:   # WAKE-UP
        self.m1.value(0); self.m0.value(1)
    elif mode == 2:   # POWER-SAVING
        self.m1.value(1); self.m0.value(0)
    elif mode == 3:   # SLEEP (CONFIG)
        self.m1.value(1); self.m0.value(1)

    time.sleep_ms(20)
    self.wait_aux()
    if self.debug:
        print(f"Mode {mode} active")
```

3. Configuration de la Communication :

3.1. Adressage du Module :

3.1.1. Principe :

3.1.2. Code :

```
def setup_fixed_transmission(self, addr_h, addr_l, channel):
    self.set_mode(3)
    # C0 + ADDR_H + ADDR_L + 1A (9600bps) + CHAN + C4 (Fixe)
    config_cmd = bytes([0xC0, addr_h, addr_l, 0x1A, channel, 0xC4])
    if self.debug:
        print(f"Envoi configuration fixe : {config_cmd.hex().upper()}")
    self.uart.write(config_cmd)
    time.sleep_ms(200)
    if self.uart.any():
        response = self.uart.read()
        if self.debug:
            print(f"Confirmation module : {response.hex().upper()}")
    self.set_mode(0)
```

3.2. Communication Module a Module :

3.2.1. Principe :

Ce mode de communication est un protocole simplex entre deux modules dont un qui choisit la destination et un autre qui ne fait qu'écouter en scrutation

3.2.2. Code :

```
def send_point_to_point_v(self, target_h, target_l, target_chan, message):
    self.wait_aux()
    header = bytes([target_h, target_l, target_chan])
    message_txt = str(message)
    self.uart.write(header + " " + message_txt)
    if self.debug:
        print(f"Envoyé à {target_h:02x}{target_l:02x} sur canal {target_chan}")
```

3.3. Communication par Canal / Broadcast :

3.3.1. Principe :

Ce mode de communication est semblable au broadcasting ip qu'on peut trouver sur les réseaux filaires , on choisit le canal par lequel on veut communiquer sans se soucier de l'adresse de destination.

le module se considère comme concernée si l'émetteur envoie un message à **son adresse**, a une adresse **0x00 00** et a une adresse **0xFF FF**

3.3.2. Code :

```
def send_broadcast(self, target_chan, message):
    self.wait_aux()
    header = bytes([0xFF, 0xFF, target_chan])
    self.uart.write(header + message.encode('utf-8'))
    if self.debug:
        print(f"Broadcast envoyé sur canal {target_chan}")
```